NORTH BENGAL INTERNATIONAL UNIVERSITY

Lab Report : 6

TOPIC : REVERSE ALPHABET PRINT WITH SPACE IN ASSEMBLY LANGUAGE

COURSE CODE : CSE 0611-3322

COURSE TITLE : MICROPROCESSOR & ASSEMBLY LANGUAGE SESSIONAL.

Submitted By:

MD. TANVIR RAHMAN PRANTO

0712340012101005

CSE -26

Submitted To:

SAIFUR RAHMAN

LECTURER, CSE

Theory: Assembly language is a low-level, human-readable programming language. It uses symbols instead of binary, mapping directly to machine instructions for specific architectures. It's ideal for high-performance or hardware-specific tasks.

Printing Characters from 'Z' to 'A' with Spaces in Assembly Language:

To print characters from 'Z' to 'A' with spaces in assembly language, start by initializing the data segment and loading the ASCII value of 'Z' into a register, such as `dl`. Use a loop to print each character followed by a space. In each iteration, decrement the character value in `dl` and output it using a DOS interrupt (21h, function 2). Continue this loop until the character 'A' is printed. Ensure to manage the loop counter and exit the program cleanly after printing all characters. This method demonstrates control flow, looping, and character manipulation in assembly language.

Code:

```
.model small
.stack 100h

.data
    alph db 'Z'    ; Start from 'Z'
    space db ' '   ; Define a space character

.code
main proc
    ; Initialize data segment
    mov ax, @data
    mov ds, ax

    ; Set loop counter for 26 letters
    mov cx, 26

level1:
    ; Display current character
    mov ah, 2
    mov dl, alph
    int 21h

    ; Display space character
    mov dl, space
    int 21h

    ; Decrement to move backward in alphabet
    dec alph
    ; Repeat loop
    loop level1

    ; End program
    mov ah, 4ch
    int 21h

main endp
end main
```

Output:

Conclusion: Through this code, we learn how to manipulate and display characters in reverse alphabetical order in assembly language. It shows the importance of initializing the data segment and using loops to handle repetitive tasks. By incorporating DOS interrupts for output, we see practical applications of low-level programming concepts and efficient character handling.